

Processes and Operating System

Introduction

- Sophisticated applications need multiple programs to be written (as single piece of code will not suffice) to perform multiple operations
- 2 fundamental abstractions: the *process* and the *operating system (OS)*
- The *process* cleanly defines the *state of an executing program*, while the *OS* provides the mechanism for switching execution between the processes.
- The need to satisfy complex timing requirements – events happening at very different rates, intermittent events, and so on – causes us to use processes and operating systems to build embedded software
- Compartmentalizing the functions and encapsulating in the OS makes it viable to meet complex timing requirements (time-bound tasks) with relatively clean control within the processes

Differences between OS and RTOS

Feature	Operating System (OS)	Real-Time Operating System (RTOS)
Definition	General-purpose system for managing computer resources	Specialized OS designed for time-critical applications
Primary Goal	Maximize throughput and user convenience	Guarantee timing deadlines
Response Time	Not deterministic (may vary)	Deterministic (predictable)
Timing Constraint	No strict timing requirement	Strict timing constraints
Task Scheduling	Fairness and efficiency-based	Priority and deadline-based
Interrupt Handling	May have latency	Minimal latency

Differences between OS and RTOS

Feature	Operating System (OS)	Real-Time Operating System (RTOS)
Multitasking	Yes (but not time-critical)	Yes (time-critical multitasking)
Examples of Tasks	File handling, web browsing	Sensor reading, motor control
Failure Impact	Delay acceptable	Delay may cause system failure
Memory Usage	Relatively high	Optimized and minimal
Examples	Windows, Linux, macOS	FreeRTOS, VxWorks, RTLinux

Multiple tasks and Multiple Processes

- The RTOS helps build more complex systems using several programs that run concurrently
- Processes and tasks are the building blocks of multitasking systems, much as C functions and code modules are the building blocks of programs
- Most embedded computing systems do more than one thing – that is, the environment can cause mode changes that in turn cause the embedded system to behave quite differently
- When designing a telephone answering machine, we can define *recording a phone call* and *operating the user's control panel* as distinct tasks – logical distinct, operates at different rates

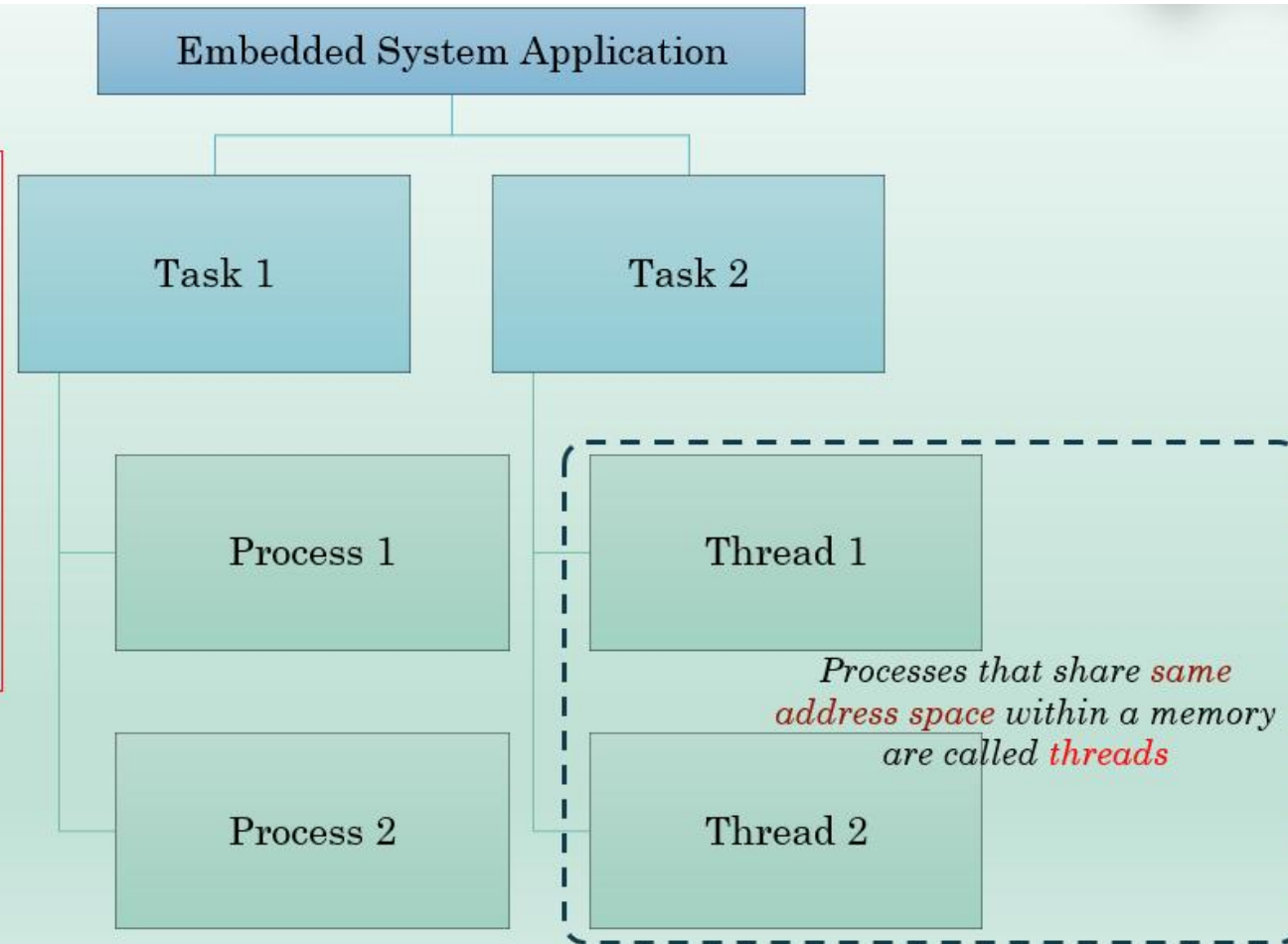
Multiple tasks and Multiple Processes

- The term **task** is used in several different ways in real-time computing. We will use it to mean a set of real-time programs that may communicate
- While a **process** is a **single execution of a program** – same program ran twice means we've created two processes

Relationship between tasks, processes (or threads)

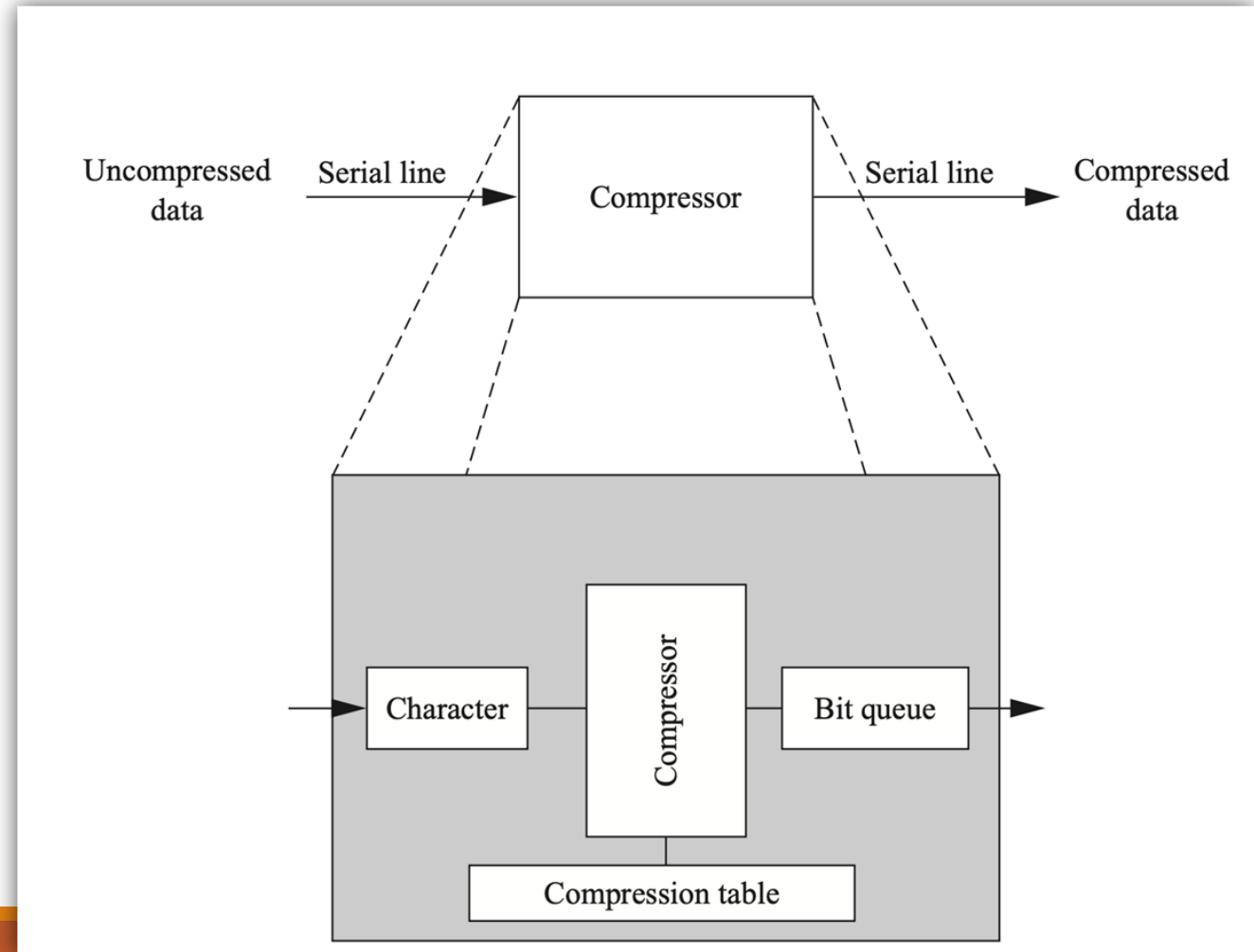
- In some OS, MMU stores each process in a separate address space
- While in others, particularly lightweight RTOSs, the processes run in the same address space, called *threads*

A Thread is a unit of execution within a process. A process can have just one threads to many threads.



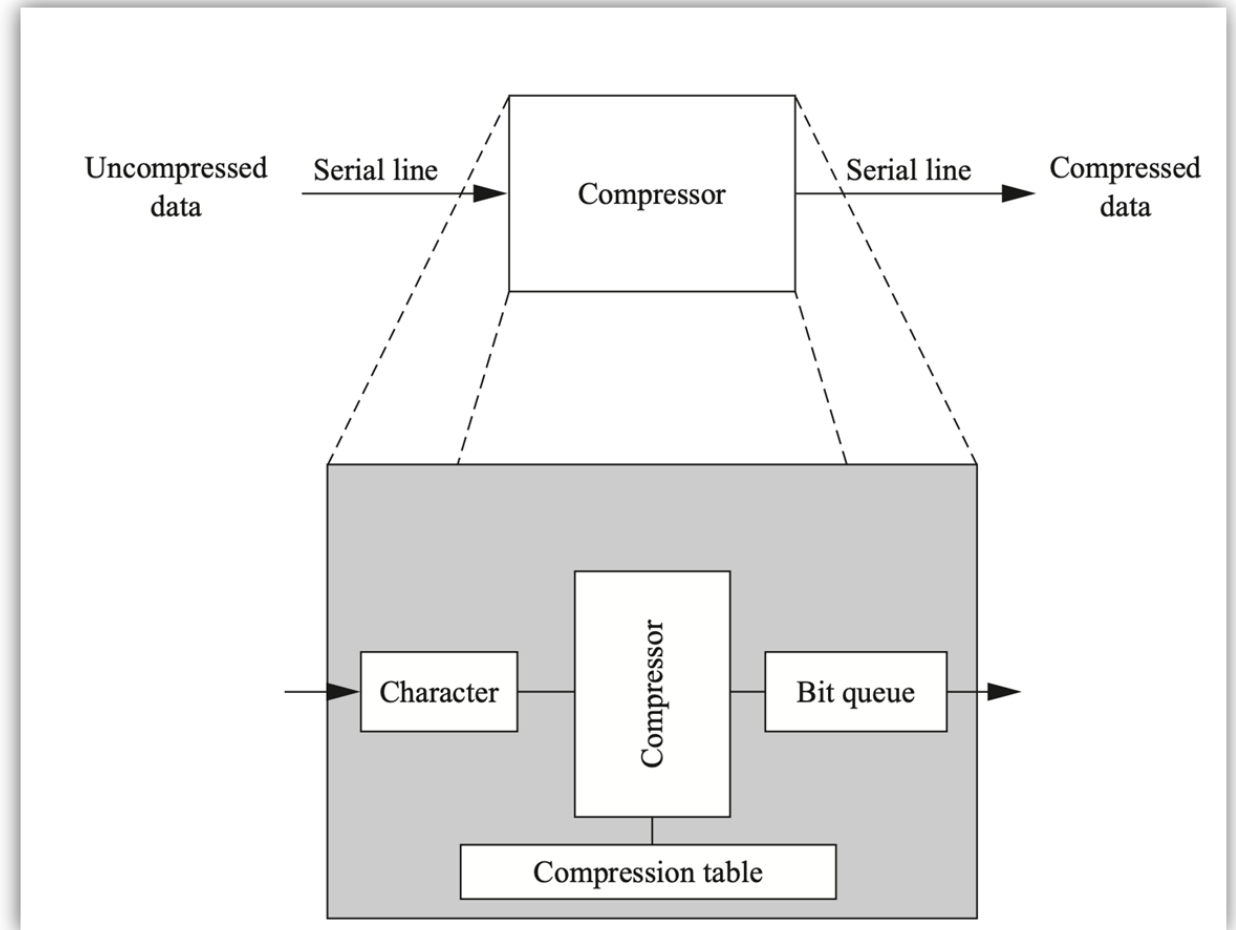
Why tasks are needed? – An illustration – compression engine

- Let's take a compression engine for an example
- The engine is connected to serial ports at both ends
- Incoming serial data arrives fast
- Compression engine will compress at a different rate, say, the program may emit 2 bits for the 1st byte and 7 bits for the 2nd
- Need: We need to send compressed data serially and slowly as 8-bit chunks



Why tasks are needed? – An illustration – compression engine

- Addedly there can be another complexity, like a push button
- A push button may be used by the user to either enable/disable compression (arbitrary and asynchronous)
- Sampling of push button is needed
- Can't be accustomed in a single code as sampling rate vary and a branching to sampling loop may be at different rate



Why tasks are needed? – contd..

- Hence,
 1. To receive all incoming bytes at high speed (rate 1)
 2. To compress and emit compressed bits at slow speed, without losing any data (rate 2)
 3. To sample the push button depression at another speed without losing any depression (rate 3)

We need to use tasks which are independent, each operating in its own rate in a multi-rate system like this.

$$\text{Baud rate} = \frac{\text{Number of bits transmitted}}{\text{second}}$$

$$\text{Data rate} = \frac{\text{Number of Compressed bits emitted}}{\text{second}}$$

$$\text{Compression ratio} = \frac{\text{Number of bits represented}}{\text{character}}$$

Multiple Rate System

- Implementing code that satisfies timing requirements is even more complex when multiple rates of computation must be handled
- Multi-rate embedded computing systems are very common, including [automobile engines, printers, and cell phones](#)
- In all these systems, certain operations must be executed periodically, and each operation is executed at its own rate
- Let's analyze the first example automobile engine

Example: Automobile engine control

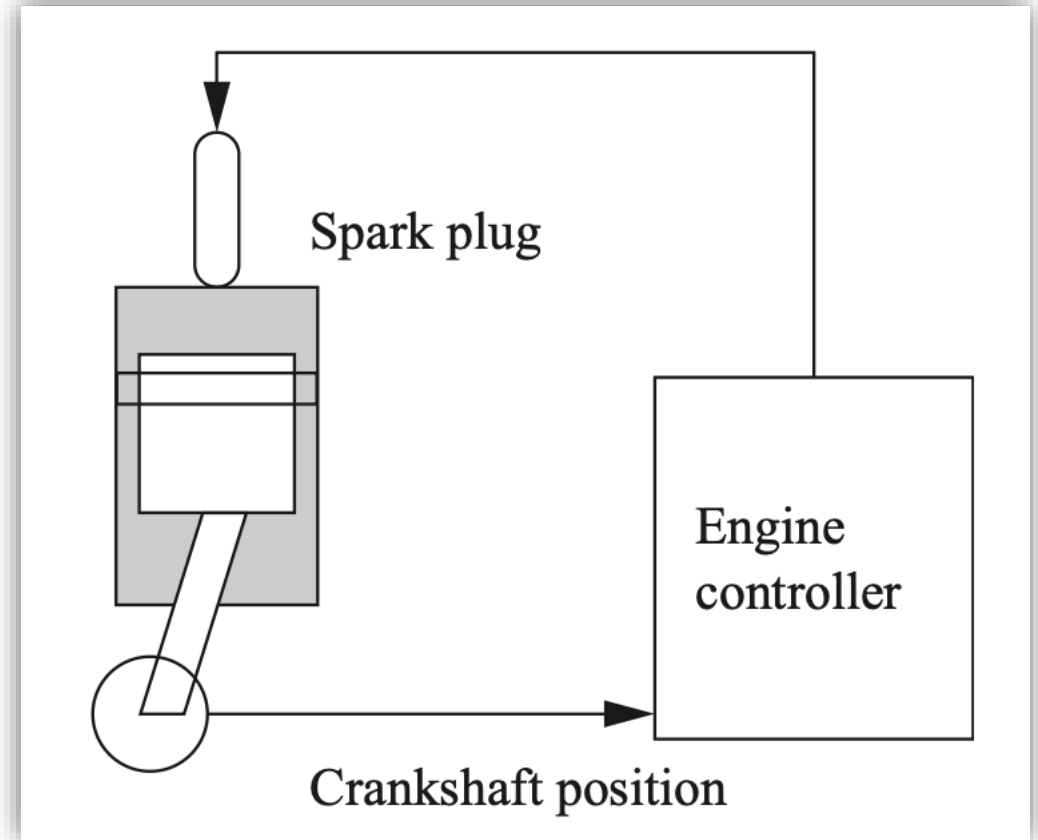
Highest task: To ignite the spark plug

Conditions:

- Abide by the Governments' regulation on emissions
- Satisfy customers in terms of:
 - Performance
 - Ease of starting in extreme weather (hot and cold)

Engine operations (mechanical)

- Spark plug ignites the air-fuel mixture
- This causes combustion which in turn moves the piston inside the engine up/down
- Up/down motion of the piston is converted to rotational motion of the wheels connected by a crankshaft



A motorcycle's engine controller

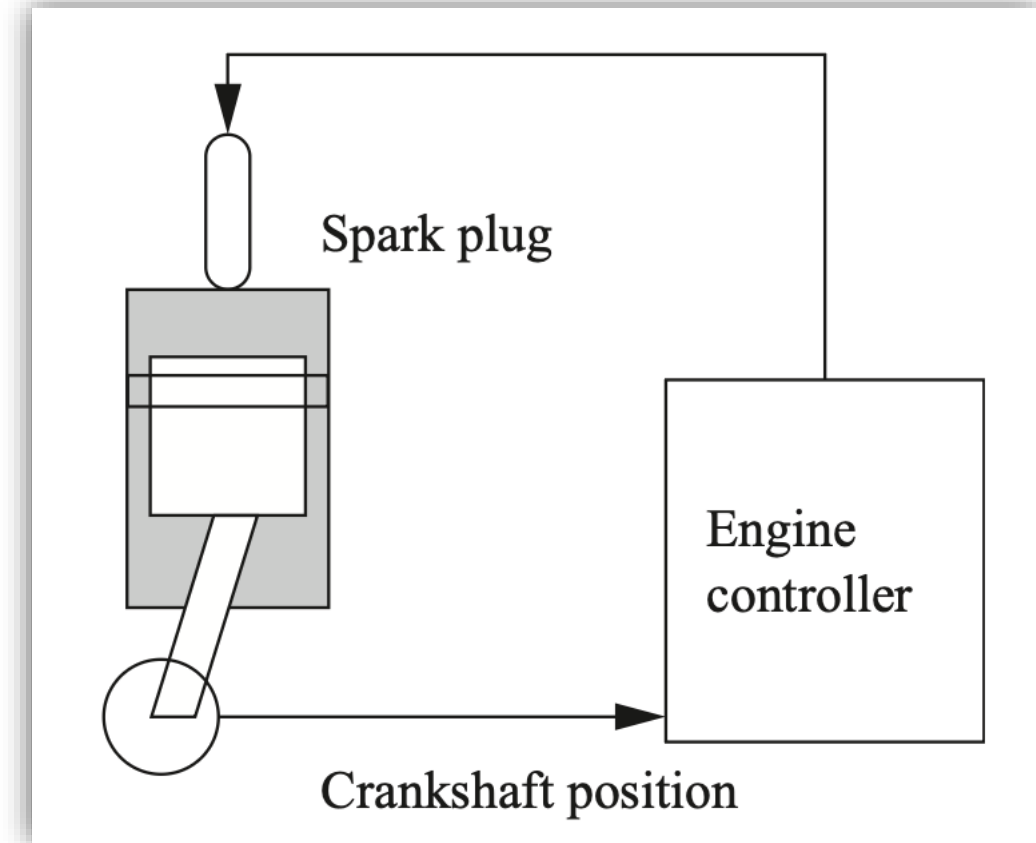
Automobile engine control

Multiple Inputs (sensors)

Sensor	Purpose
Crankshaft	Engine speed
Throttle (gas pedal)	Driver demand
Oxygen sensor	Emission control
Temperature	Engine condition

Multiple Outputs

Outputs	Function
Spark plug	Ignition
Fuel injection	Air-fuel ratio



A motorcycle's engine controller

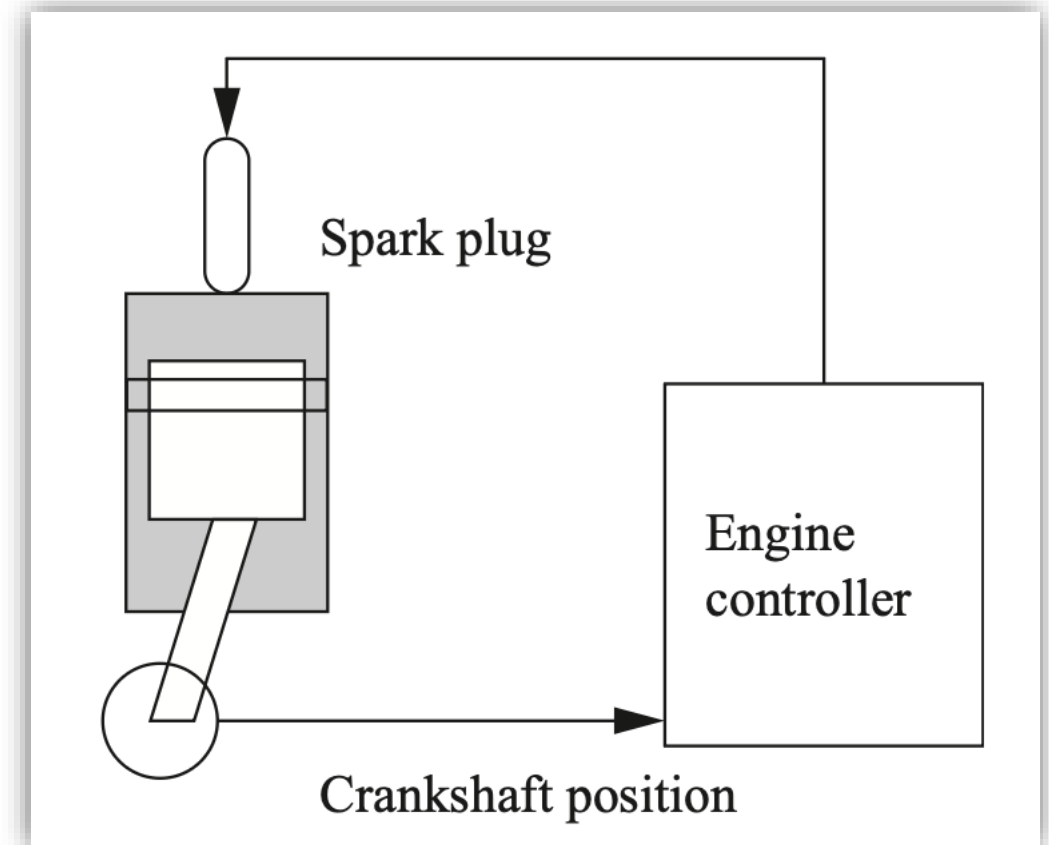
Automobile engine control

Multi-mode control scheme should control during

Mode	Description
Warm-up	Cold start
Cruise	Normal driving
Hill climb	High load

Here each mode operates at different time duration

Variable	Time to move full	Update period (ms)
Engine spark timing	300	2
Throttle	40	2
Air flow	30	4
Battery voltage	80	4
Fuel flow	250	10



A motorcycle's engine controller

Tasks run at different speeds

Task	Rate
Spark firing	Very fast
Crankshaft sensing	Fast
Throttle sensing	Medium
Oxygen sensor	Slow

1. Fastest task takes 2 ms
2. Slowest rate is 4 s
3. A range in 3x magnitude

- Each task run at its own pace
- If we write,

```
while(1)
{
    read_crank();
    fire_spark();
    read_throttle();
    read_oxygen();
}
```

- Slow tasks delay fast tasks
- Fast tasks miss deadlines

Consequence of a single code & solution

Consequences:

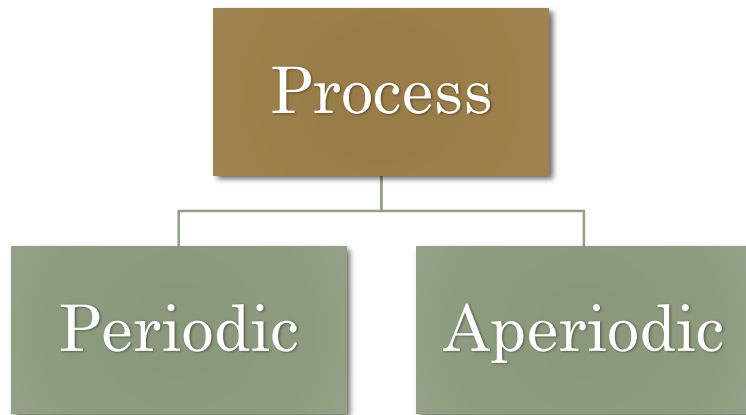
- ✗ Engine misfires
- ✗ Poor performance
- ✗ Emission failure

Solution:

- Designing a multi-rate system
 - Task 1: Spark firing (few ms)
 - Task 2: Crankshaft sensing (fast)
 - Task 3: Throttle reading (medium)
 - Task 4: Oxygen sensor (slow)

Without tasks	With tasks
One loop tries to handle everything	Each activity runs independently
Timing becomes unpredictable	Timing is controlled
System fails	System is reliable

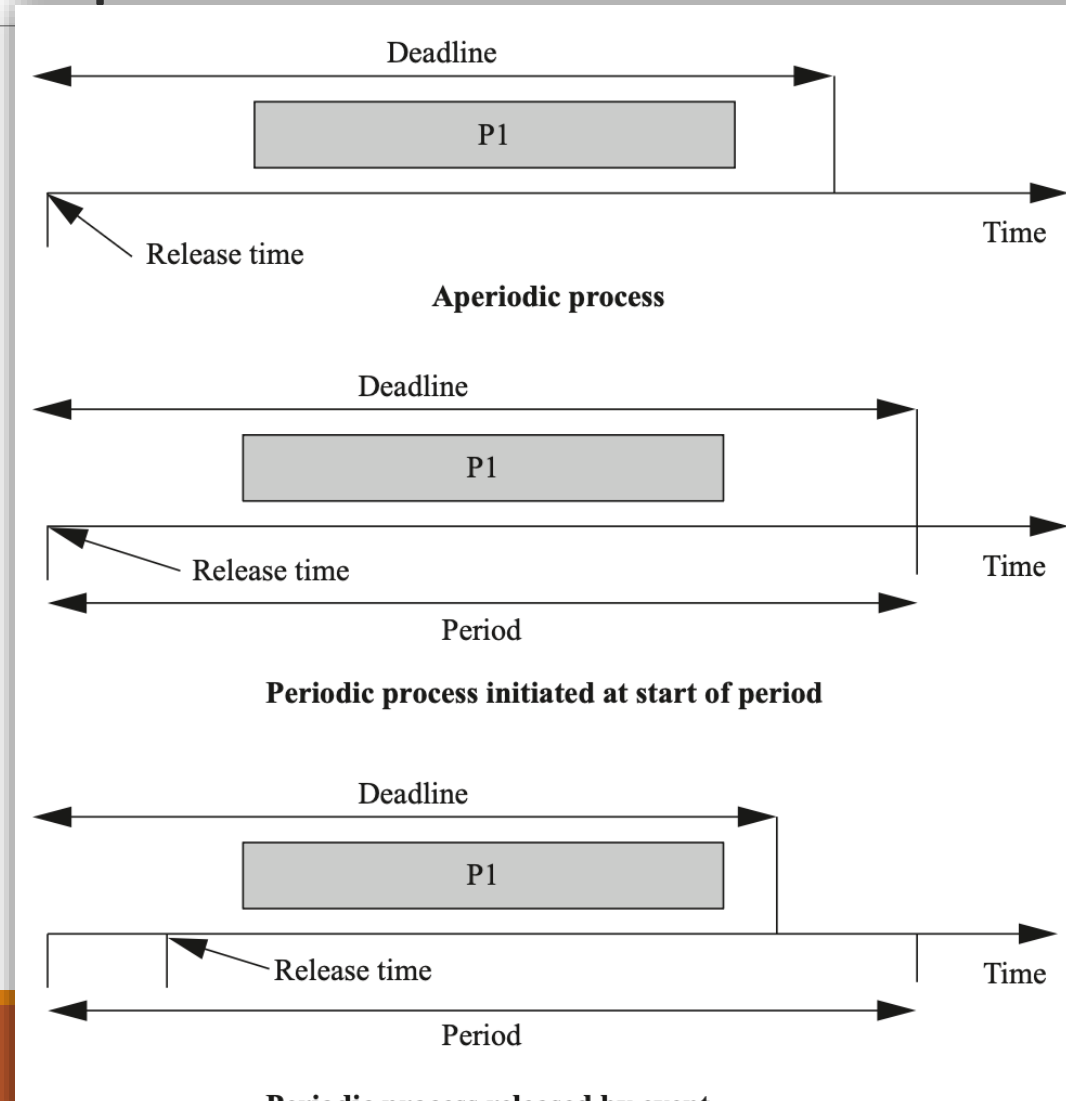
Timing requirements on a processes



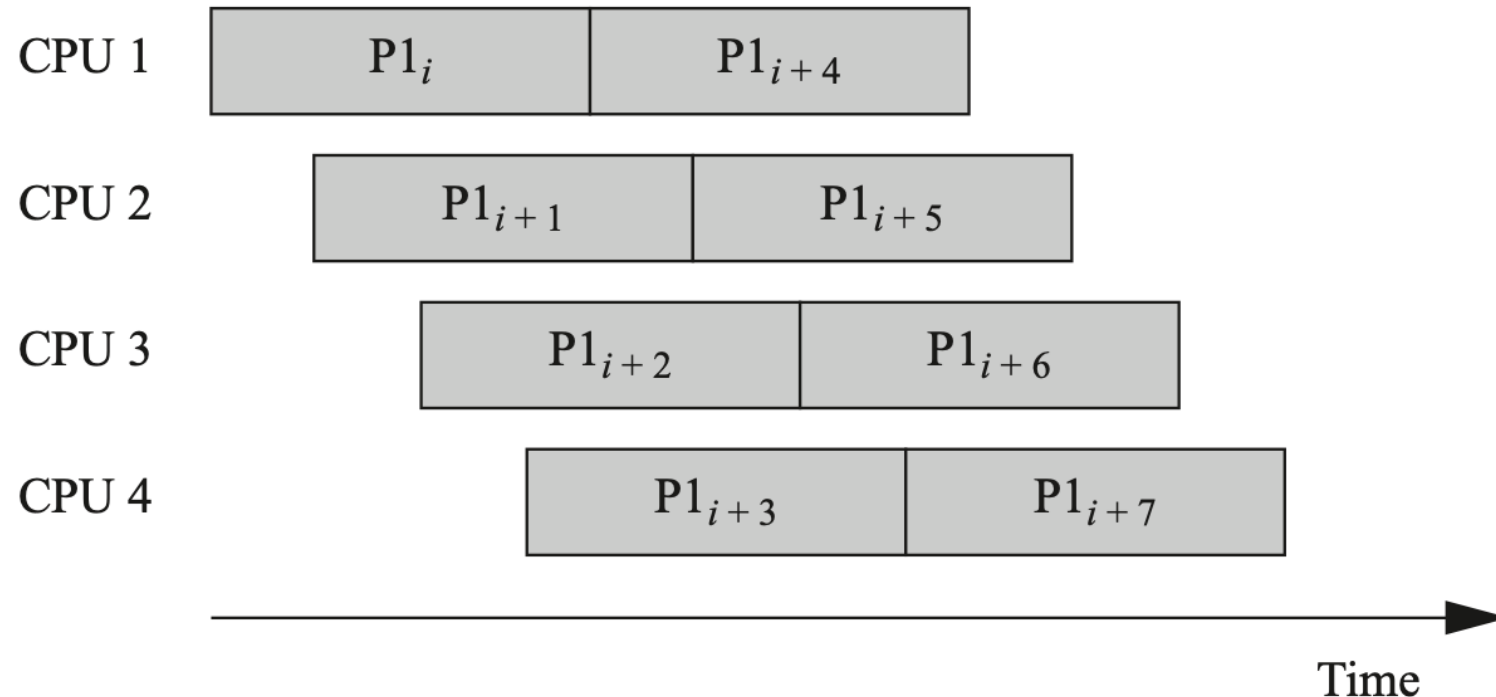
- ❖ Process: A unit of execution with timing requirements
- ❖ Two important requirements on processes: **initiation time** and **deadline**
- ❖ **Initiation time**: Time at which process actually start executing on the CPU.
- ❖ **Deadline**: Is the time by the process should have completed
- ❖ **CLASSIFICATION**:
- ❖ **Periodic process**: One which repeats at regular intervals of time
- ❖ **Aperiodic process**: Event driven, a completion of a previous event or arrival of a data

Timing requirements on a processes

- Period (T): Time between successive executions of a process
- Initiation interval: Time b/w successive executions
- Normally in non pipelined systems, initiation interval = Period (T)
- In pipelined systems, initiation interval < T
- Response Time = Completion time – initiation time (or)
- Response Time = Execution time + Waiting time
- **Jitter** = Variation in execution time or completion time
- **Response time $\leq$ Deadline** (if > deadline, called deadline miss)



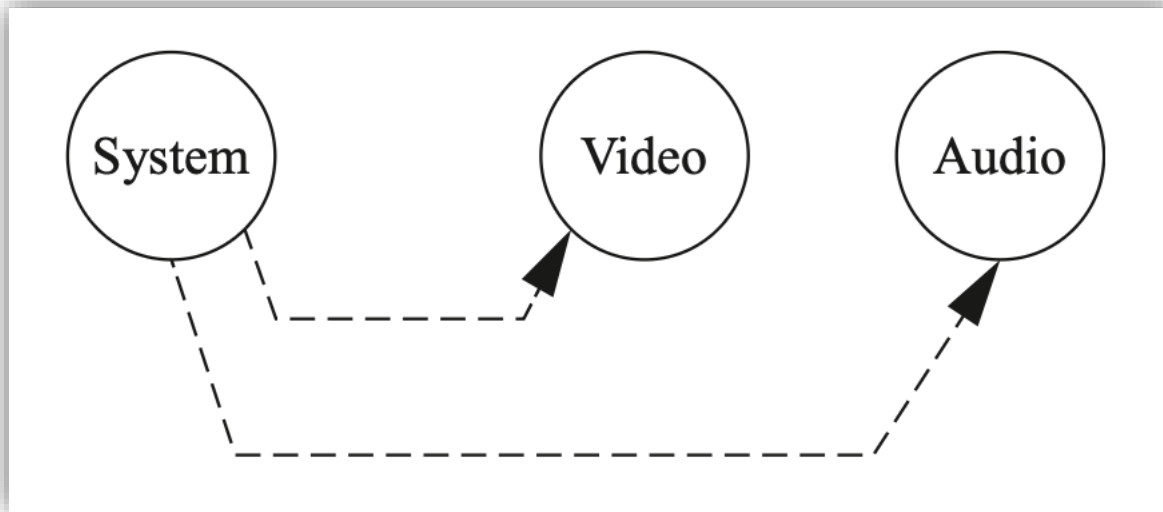
A sequence of processes with a high initiation rate



- The fig. is a classical example of pipelined execution of processes
- Here $P1$ is executed '8' times as different instances
- 4 CPU cores are available to run successive instances w/o depending on previous execution
- Hence, before completion of the currently executing instance, the next core takes care of the subsequent instance

CPU usage metric

$$\text{Utilization, } U = \frac{\text{CPU time for useful work}}{\text{Total available CPU time}}$$



Task	Function	Execution Time (C _i)	Period (T _i)
T ₁ (System)	Control & coordination	2 ms	10 ms
T ₂ (Video)	Frame processing	8 ms	40 ms
T ₃ (Audio)	Audio buffering	5 ms	20 ms

$$U = \sum \frac{C_i}{T_i}$$

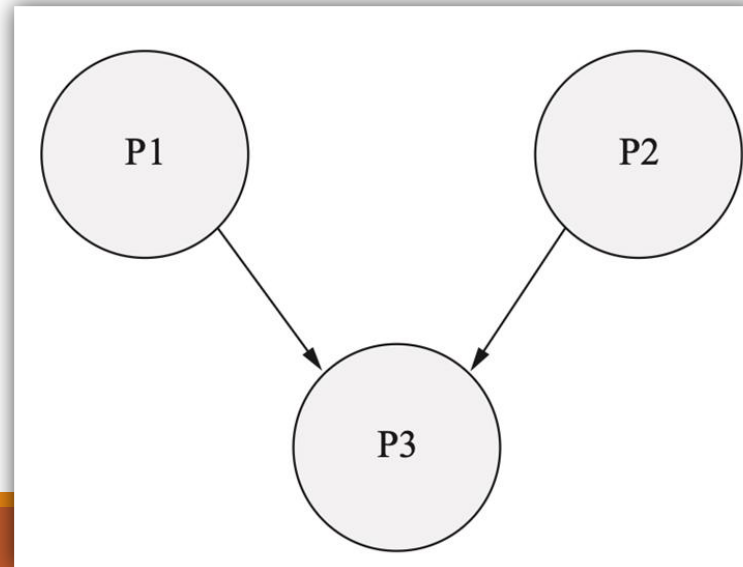
- $U = \frac{2}{10} + \frac{8}{40} + \frac{5}{20}$
- $U = 0.2 + 0.2 + 0.25 = 0.65$
- $U = 65\%$
- Normally, $0 < U < 1$
- $0 \rightarrow$ poor 'U'
- $1 \rightarrow$ good 'U' design
- Hyperperiod = **LCM (10, 20, 40)**
- Hyperperiod = 40 ms

Summary of timing requirements

Concept	Meaning
Period	How often task repeats
Rate	Frequency of execution
Initiation time	When task starts
Deadline	When task must finish
Response time	Actual completion time
Jitter	Timing variation
Hyperperiod	Repetition cycle of schedule

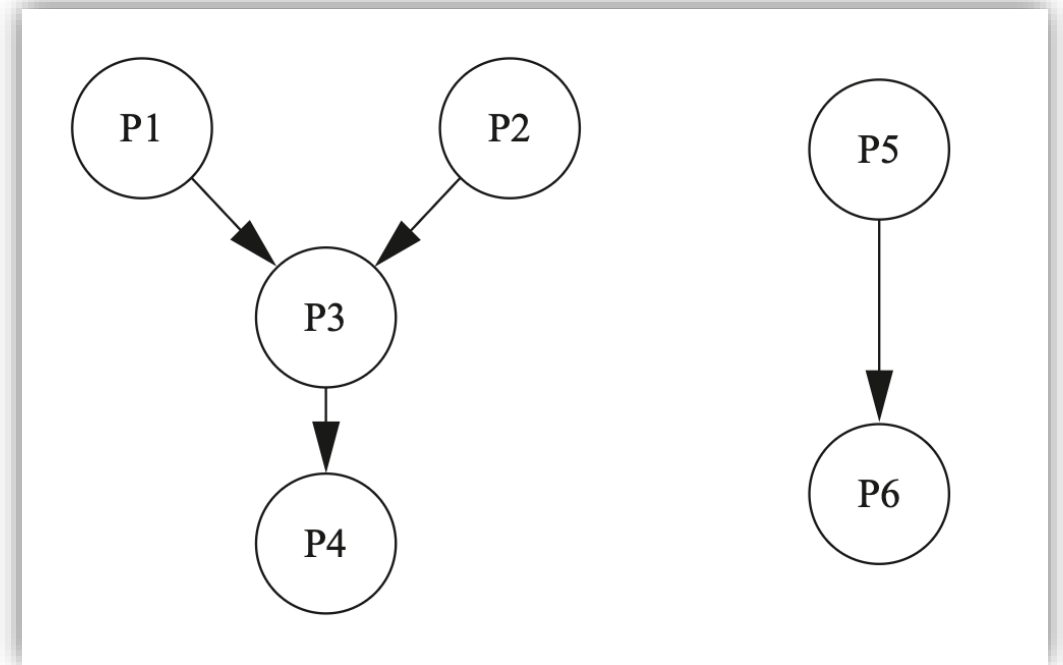
A task graph

- The set of processes with data dependencies is known as task graph.
- The diagram is a representation of a task with 3 subtasks
- Arrows in the task graph show data dependencies
- P3 can't start until it receives the results of P1 and P2; while P1 & P2 can execute in parallel



Timing requirements – Data dependencies

- P1, P2, P5, are independent
- P3, P4, P6 are dependent
- Can't make a cycle because of interdependencies & swap operations



Directed Acyclic Graph (DAG)

Running periodic processes: Scenario 1

- ❖ Imagine, a task comprises of periodic processes ideally of different rates
- ❖ Assume, at first their rates are equal (to understand the working of a normal conditional)

```
while (TRUE) {  
    p1();  
    p2();  
}
```

- ❖ Inference: Here, an infinite loop is created; and it runs as quickly as possible without consideration of period
- ❖ This is a disadvantage in terms of periodic processes, which necessarily have to repeat only after a period 'P'

Running periodic processes: Scenario 2 (Refinement 1)

- ❖ Now, let's refine the previous loop to run at constant rate by using a timer loop (using timer)
- ❖ Timer(s) shall be used to create periodic interrupts
- ❖ Hence, only when timer interrupt occurs, the processes run; else nil

```
void poll() {  
    p1();  
    p2();  
}
```

- ❖ Inference: Timer interrupt triggers this loop – both processes run once
- ❖ **Disadvantage(s)**:
 - ❖ Processes may run too long, while running processes, interrupts would have been disabled so the subsequent run of the loop may be delayed (making it aperiodic instead of periodic)

Running periodic processes: Scenario 3 (Refinement 2)

- Our problem is to execute different processes at different rates
- Proposal
 - We may dedicate one timer unit for one rate; hence multiple timers shall be used
 - We could then use a function to collect all the processes that run at that rate
- Inference:
 - This works, but it does require multiple timers
- **Disadvantage(s)**:
 - We may run out of timer units

```
void pA() {  
    /* processes that run at rate A */  
    p1();  
    p3();  
}  
void pB() {  
    /* processes that run at rate B */  
    p2();  
    p4();  
    p5();  
}  
  
...
```

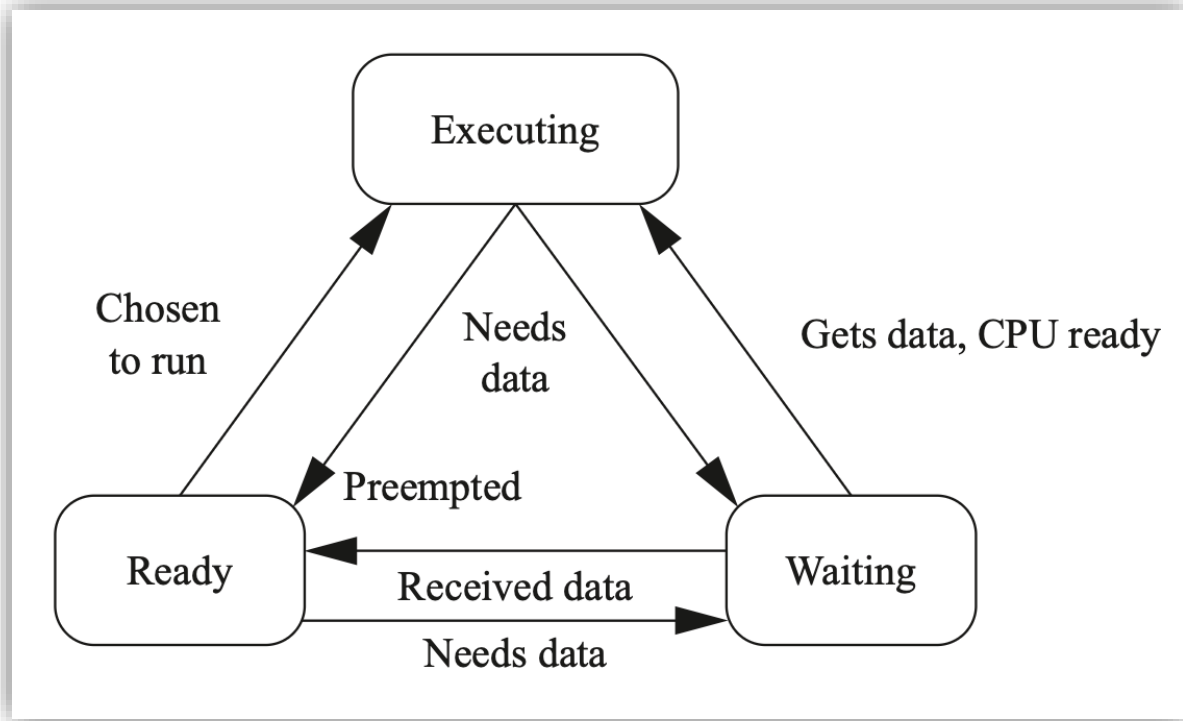
Running periodic processes: Scenario 4 (Refinement 3)

- To address the timers crunch, we may use counters to divide the counter rate
- Assume, for e.g., process $p2()$ must run at $1/3$ the rate of $p1()$; then we can use this code

```
static int p2count = 0; /* use this to remember count across timer
interrupts */
void p1() {
    p1();
    if (p2count >= 2) { /* execute p2() and reset count */
        p2();
        p2count = 0;
    }
    else p2count++; /* just update count in this case */
}
```

- This solution allows us to run processes at multi-rates that are multiples of one another,
- **Disadvantage:**
 - ✗ However, when they're not relatable, counting becomes complex and buggy one.

Process state and scheduling



Scheduling states of a process

- **Scheduling** is the OS function that selects which process runs next from the ready queue
- A process exists in three states: **waiting**, **ready**, and **executing**, with only one executing at a time
- Processes move to **waiting** for data/timer, to **ready** when conditions are met, and to **executing** when scheduled
- A **scheduling policy** determines selection strategy and directly affects system **timing guarantees** and **CPU utilization**
- There is a trade-off: **higher utilization often requires more complex policies**, which increase **scheduling overhead**

Scheduling algorithms

- FCFS
- Preemptive task scheduling
- Priority based scheduling
- Rate-Monotonic Scheduling (RMS)
- Earliest Deadline First (EDF) scheduling
- Round-robin algorithm

First Come First Serve (FCFS) Scheduling

- ❖ Basic Principle: Also known as Run-To-Completion, FCFS executes tasks in the exact order they enter the READY queue
- ❖ No priorities
- ❖ Non-Preemptive Nature
- ❖ Queue Design
- ❖ Fairness and Penalties
- ❖ Performance metrics:
 - ❖ *Generally slower*
 - ❖ *Throughput can be lower*
- ❖ Also used in **bus arbitration**
- ❖ Problem: Single master can maintain control of the bus indefinitely without intervention of the arbitrator

FCFS (Example)

Process	Duration	Oder	Arrival Time
P1	24	1	0
P2	3	2	0
P3	4	3	0

Gantt Chart :



P1 waiting time : 0

P2 waiting time : 24

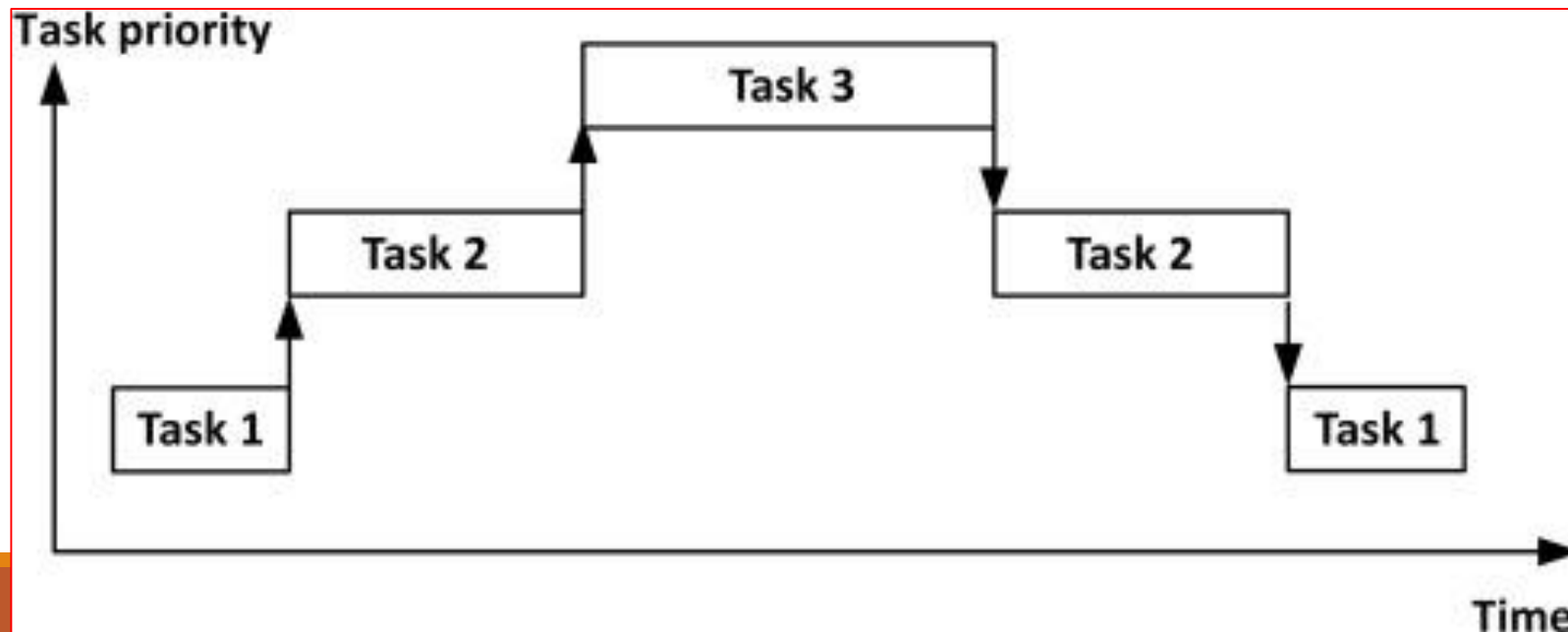
P3 waiting time : 27

The Average waiting time :

$$(0+24+27)/3 = 17$$

Preemptive task scheduling

- The tasks are prioritized and the task with the highest priority among all other tasks gets the CPU time
- Low priority task in running in CPU → Higher priority task arrives (becomes ready to run) → Kernel saves the context of the current task → Switches to the high priority task until it becomes non computable → resumes low priority task



Priority based scheduling

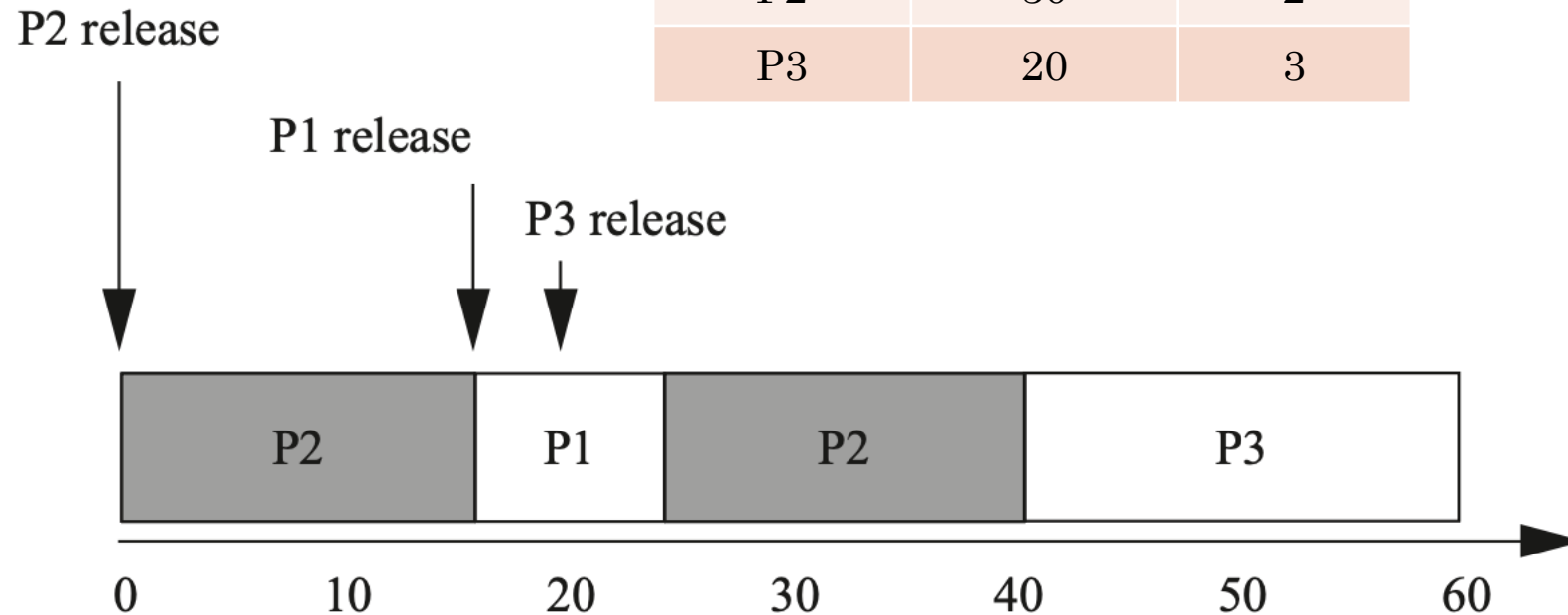
- Each process is assigned an integer-valued **priority**
- The scheduler always selects the ready process with the **highest-valued priority** for execution
- A process continues its execution until it either completes or is **preempted** by a higher-priority process
- Static priority: e.g. RMS (Rate-Monotonic Scheduling)
- Dynamic priority: e.g. EDF (Earliest Deadline First)

Process	Burst time	Priority
P1	10	1
P2	30	2
P3	20	3

Priority-Driven Scheduling

Process	Execution time	Priority
P1	10	1
P2	30	2
P3	20	3

We assume P2 is ready to run at the system start, P1 is released at time 15 and P3 is released at time 18



Priority-Driven Scheduling

- Two major ways to assign priority:
 - ❖ **Static Priority**: do not change during the execution
 - ❖ **Dynamic Priority**: Priorities that change
- Static priority: e.g. RMS (Rate-Monotonic Scheduling)
- Dynamic priority: e.g. EDF (Earliest Deadline First)

Rate-monotonic scheduling [RMS]

❖ **Rate Monotonic Scheduling (RMS)** is a **priority-based scheduling algorithm** used for **periodic tasks** in real-time systems

❖ Tasks that run more frequently get higher priority

❖ Shorter period → high priority

❖ Longer period → low priority

Task	Period	Priority
T1	10 ms	Highest
T2	20 ms	Medium
T3	50 ms	Lowest

❖ A set of processes can be scheduled using RMS only if they satisfy the following equation:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n(2^{1/n} - 1)$$

❖ Where 'n' is the number of processes, C_i is the computation time of the process, T_i is the Time period for the process to run and U is the processor utilization.

RMS *contd..*

Problem

1. Consider an application with 3 tasks refer Table below, and check if RMS is applicable for their scheduling.

Task	Execution Time (C_i)	Period (T_i)
T_1	2 ms	10 ms
T_2	3 ms	15 ms
T_3	5 ms	30 ms

Solution: Here, $n = 3$,

$$U = \sum \frac{C_i}{T_i} = \frac{2}{10} + \frac{3}{15} + \frac{5}{30} = 0.56$$

As per condition, $n(2^{1/n} - 1) = 0.78$ i.e. if $0.56 \leq 3(2^{1/3} - 1)$; so RMS is schedulable. So we can assign the priorities in the following order:

$$T_1 > T_2 > T_3$$

RMS *contd..*

Here is a simple set of processes and their characteristics.

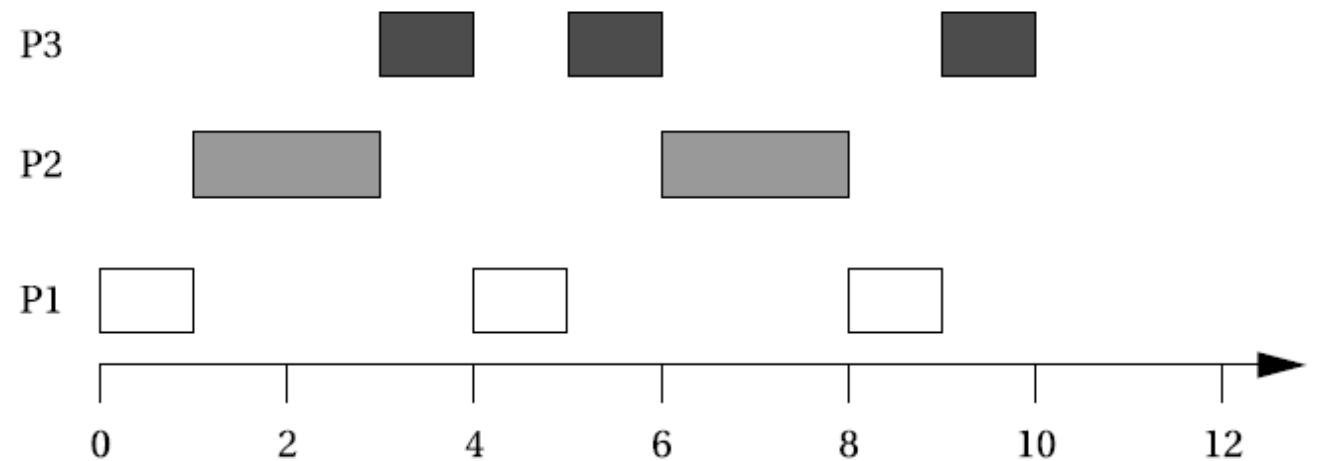
Process	Execution time	Period
P1	1	4
P2	2	6
P3	3	12

❖ Hyperperiod=12

❖ Priority Order:

P1>P2>P3

❖ Scheduling of P1,P2,P3 using RMS:



Gantt Chart of the Scheduling of Processes

Time

Conditions:

The analysis of Rate monotonic scheduling assumes few properties that every process should possess. They are :

- Processes involved should not share the resources with other processes.
- Deadlines must be similar to the time periods. Deadlines are deterministic.
- Process running with highest priority that needs to run, will preempt all the other processes.
- Priorities must be assigned to all the processes according to the protocol of Rate monotonic scheduling.

Advantages:

- It is easy to implement.
- If any static priority assignment algorithm can meet the deadlines then rate monotonic scheduling can also do the same. It is optimal.
- It consists of a calculated copy of the time periods, unlike other time-sharing algorithms as Round robin which neglects the scheduling needs of the processes.

Disadvantages:

- It is very difficult to support aperiodic and sporadic tasks under RMA.
- RMA is not optimal when the task period and deadline differ.

Earliest Deadline First [EDF]

- ❖ Earliest Deadline First (EDF) is an optimal dynamic priority scheduling algorithm used in real-time systems.
- ❖ It assigns priorities to the task according to the absolute deadline.
- ❖ The task whose deadline is closest gets the highest priority. The priorities are assigned and changed in a dynamic fashion.
- ❖ EDF is very efficient as compared to other scheduling algorithms in real-time systems.
- ❖ It can make the CPU utilization to about 100% while still guaranteeing the deadlines of all the tasks.
- ❖ Flexibility: EDF can handle both periodic and aperiodic tasks

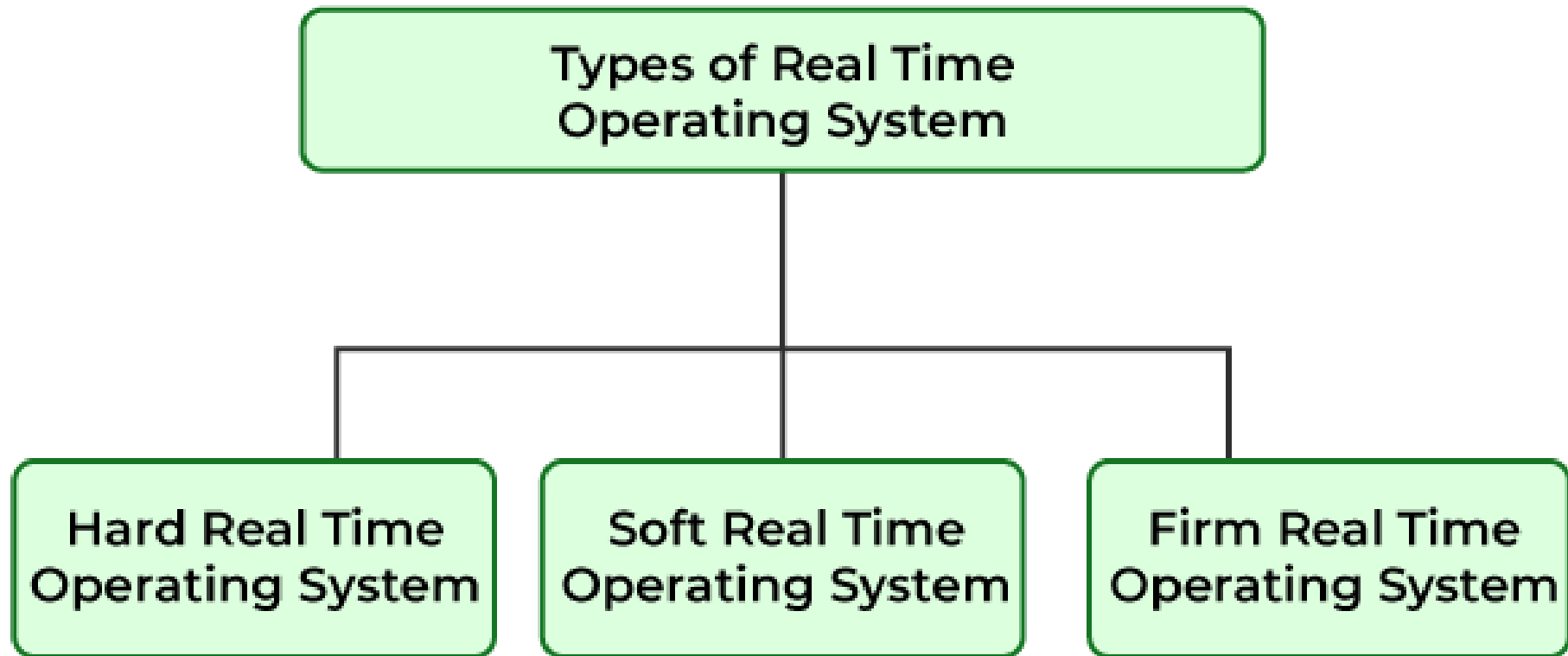
Comparison between RMS and EDF

Rate Monotonic Scheduling	Deadline Monotonic Scheduling
In Rate monotonic Scheduling, priorities are assigned to tasks according to the shortest cycle time	In Deadline monotonic Scheduling, priorities are assigned to tasks according to the shortest deadline.
Implementation of Rate monotonic scheduler is simple and can be executed using FIFO (First-in First-out) scheduling.	Implementation of Deadline monotonic scheduler is complicated and is time-taking process.
The choice of priority is decided based on the computation time or active frequency.	The choice of priority is decided based on which task is ready to be executed first.
Priority is inversely proportional to the cycle time.	Priority is inversely proportional to the deadline (time limit).
RMA is not optimal with periodic tasks when deadline and period differ.	DMA is more proficient than RMA when deadline and period differ.

Round Robin scheduling

- A common scheduling algorithm in general-purpose operating systems is **round-robin**
- **Fair chance for all processes to be executed equally; but it does not guarantee completion time of any task**

Classification of RTOS



Classification of RTOS

Real Time OS can be broadly classified into two categories:

- ❑ **Soft real time systems** - where the output of the system is acceptable even if it is delayed but as the delay increases the output quality is considered to be degraded
 - ❖ Ex: responses from a web server.
- ❑ **Hard real time systems** - These have very precise time constraints and any output after that time limit is just not acceptable
 - ❖ Ex: Control systems in nuclear reactors of flight control systems, braking systems etc.